VIT BHOPAL UNIVERSITY

Bachelors of Technology
Computer Science Engineering
(AI&ML)
Open Source Software
(CSE0002)

Lab Report

The Open Source Audit — Python

SUBMITTED TO:	SUBMITTED BY:
Dr. ADARSH PATEL School of Computing VIT Bhopal University	KUNAL KUMAR MISHRA 24BAI10443 31 March 2026

Introduction

The open source movement has revolutionized software development, distribution, and maintenance worldwide. Unlike proprietary software, where source code is kept secret and controlled by a single firm, open source software is developed, distributed, and enhanced in public, with source code freely available and enhanced by many developers worldwide. This report will discuss one of the most influential open source software available in the world today: Python.

This software was not developed by a firm with a business motive in mind. It was developed by a curious individual with a motive to make a better tool and give it back to society for free. This audit will not only discuss what this software does, but why it was made, what liberties its license takes, how it lives on a Linux system, and how it compares to other proprietary software.

Along with this written audit, five shell scripts will be designed and tested on Ubuntu Linux (WSL2) to show proficiency in using commands and a basic understanding of how open source software works in a real-world environment.

Part A — Origin and Philosophy

A1. The Problem Python Was Created to Solve

In the late 1980s, Guido van Rossum was a programmer working at CWI — Centrum Wiskunde & Informatica — a research institute in the Netherlands. He was experienced with a teaching language called ABC, which was elegant and easy to use, but too limited and closed for real-world programming work. He was also familiar with C and Unix shell scripting, but found that C was too complex and low-level for rapid development, while shell scripts were too limited for anything serious.

Van Rossum wanted a language that sat in the middle — something powerful enough for real tasks, but simple and readable enough that a programmer could understand code written by someone else without hours of study. In December 1989, during the Christmas holiday period, he began working on a new interpreter as a personal project to keep himself busy. He named it Python, not after the snake, but after the British comedy series Monty Python's Flying Circus, which he enjoyed.

What made Python's origin story different from many other programming languages was the decision to make it open source from the very beginning. Van Rossum was not working for a company trying to sell a product. He was a researcher who believed that good tools should be available to everyone. He released Python 0.9.0 publicly in 1991, sharing the source code freely so that anyone could read it, use it, improve it, or build upon it.

Before Python existed, someone who wanted to write a quick automation script had to either struggle with complex C code, rely on limited shell scripting, or use expensive proprietary tools. Python filled that gap with a language that was free, readable, and powerful. This is why Python's creator is often quoted as saying that code should be readable like English — because readability was not an afterthought but the central design philosophy from day one.

Today, Python powers everything from web servers and data science pipelines to artificial intelligence research and space exploration tools. NASA, Google, Instagram, and thousands of universities use Python daily. None of that would have been possible if van Rossum had kept the source code private or sold it to a corporation.

A2. The License — What It Actually Says

Python is released under the Python Software Foundation License (PSF License), which is an open source license approved by the Open Source Initiative. The PSF License is a permissive license, meaning it places very few restrictions on what users can do with the software. To understand what this means in practice, it is important to first understand the four freedoms of free software as defined by the GNU Project.

The four freedoms are: the freedom to run the program for any purpose; the freedom to study how the program works and modify it; the freedom to redistribute copies; and the freedom to distribute copies of modified versions. The PSF License grants all four of these freedoms. Users can use Python for any purpose — commercial or personal. They can read the source code and change it. They can share copies. And they can release their own modified versions.

One important question about any open source license is whether a company can take the software, modify it, and sell it without sharing their changes. Under the PSF License, the answer is yes — a company can do this. Unlike the GPL (General Public License), the PSF License does not have a copyleft requirement. This means there is no obligation to release modified source code back to the public. This makes Python's license more business-friendly than GPL-licensed software.

The difference between GPL and MIT (or PSF) licensing is significant. GPL is a copyleft license — if you distribute software that uses GPL code, you must also release your source code under the GPL. This ensures that open source remains open. MIT and PSF licenses are permissive — they allow proprietary use without requiring source disclosure. If building something intended to share with the community, I would choose GPL to ensure improvements stay open. If building something for commercial deployment where flexibility was needed, I would consider MIT or PSF.

The distinction between 'free as in free beer' and 'free as in freedom' is crucial to understanding open source philosophy. 'Free beer' means at no cost. 'Free as in freedom' means you have the rights to use, study, modify, and share. Python is both — it costs nothing to download, and it grants genuine freedoms to its users. A proprietary language might be free to download but still restrict what you can do with it — making it free beer but not free as in freedom.

A3. The Ethics of Open Source

The question of whether all software should be open source is genuinely complex. The argument in favour is compelling: software has become infrastructure. Roads, water systems, and electricity grids are public goods because society depends on them. Modern digital infrastructure — operating systems, web servers,

databases, programming languages — is equally critical, and restricting access through proprietary licensing creates inequality. People in wealthier countries or larger companies can afford expensive licenses; smaller organisations and individuals in developing economies cannot. Open source levels that playing field.

The argument against mandatory open source is equally valid. Software development requires significant investment of time, skill, and resources. Companies and individuals need sustainable business models to continue building good tools. If every piece of software had to be open source, there would be less commercial investment in software development, potentially resulting in fewer high-quality tools. Proprietary software also sometimes offers better support, accountability, and reliability guarantees than community-maintained alternatives.

The ethics of companies profiting from community labour is a genuinely contested issue. Red Hat built a billion-dollar business on Linux — software created by thousands of volunteer contributors. Google's servers run on Linux. Meta's infrastructure depends on open source tools. Is this exploitation? One view says yes — corporations extract enormous value from work done for free by community members who receive nothing in return. The other view is that these companies also contribute back — Red Hat employs kernel developers, Google contributes to countless open source projects, and their commercial success funds infrastructure that benefits everyone.

The phrase 'standing on the shoulders of giants' means that every new creation builds on the work of those who came before. In software, this is literally true — Python itself was built on C, which was built on assembly language, which was built on decades of computer science research. Open source enables this chain of innovation to continue freely. Rather than every developer reinventing the wheel, open source allows each generation to start from where the last one ended.

Part B — Linux Footprint

Python is one of the most deeply integrated pieces of software in any modern Linux system. Many Linux distributions install Python by default because core system tools depend on it. On Ubuntu 24.04 LTS — the system used for this audit — Python 3.12.3 was found to be pre-installed.

Installation Method

Python is available through Ubuntu's official package manager, APT. Installation requires only a single command: sudo apt install python3. The package is maintained by Canonical and the Ubuntu community, who track upstream releases from the Python Software Foundation and package them for Debian-based systems. For developers who need the very latest version, Python can also be compiled from source by downloading the tarball from python.org.

Key Directories

On a standard Ubuntu installation, Python's binary is located at /usr/bin/python3. The standard library modules are stored in /usr/lib/python3, with version-specific files in subdirectories such as /usr/lib/python3/dist-packages. Configuration files may be found in /etc/python3. User-installed packages via pip are typically stored in ~/.local/lib/python3.x/site-packages.

User and Group

Python itself does not run as a dedicated system user — it runs as whichever user invokes it. This is an important security consideration. When Python is used in web servers or automated pipelines, best practice is to create a dedicated low-privilege user account and run Python processes under that account. This limits the damage that a compromised Python application can cause. Running Python as root is strongly discouraged.

Service Management

Python scripts can be run as system services using systemd, Ubuntu's init system. A service file is created in /etc/systemd/system/, specifying the script to run, the user to run it as, and restart behaviour. The service can then be managed with: sudo systemctl start myservice, sudo systemctl stop myservice, sudo systemctl enable myservice, and sudo systemctl status myservice.

Update Model

Security patches for Python reach Ubuntu systems through the APT package manager. When the Python Security Response Team identifies a vulnerability, a patched version is released and Canonical packages it for Ubuntu. Users running sudo apt update followed by sudo apt upgrade will receive security patches automatically. This model means that open source security is transparent — vulnerabilities are disclosed publicly and patches are reviewed in the open.

```
admin_kunal@LAPTOP-H4DR3AJS:~/oss-audit$ python3 --version
Python 3.12.3
admin_kunal@LAPTOP-H4DR3AJS:~/oss-audit$ which python3
/usr/bin/python3
admin_kunal@LAPTOP-H4DR3AJS:~/oss-audit$
```

Part C — The FOSS Ecosystem

Dependencies

Python does not exist in isolation. It depends on a stack of other open source tools to function. The Python interpreter itself is written in C and requires GCC (the GNU Compiler Collection) to compile from source. The zlib library provides compression support. OpenSSL provides cryptographic functions that underpin Python's ssl module and secure network communications. The readline library enables interactive terminal editing in the Python REPL. All of these are open source projects with their own communities and histories.

What Python Has Enabled

Python's impact on the software ecosystem is difficult to overstate. The Django and Flask web frameworks, both open source, have powered millions of websites. NumPy and SciPy created the foundation for scientific computing in Python. Pandas transformed data analysis. TensorFlow and PyTorch — both using Python as their primary interface — have become the dominant platforms for machine learning research worldwide. Jupyter Notebooks changed how scientists share and present computational work.

Python has also influenced other languages. Languages like Ruby and Swift borrowed ideas from Python's design philosophy. The Conda package manager, designed for scientific Python, has expanded into a general-purpose environment management tool. Anaconda, a company built entirely around open source Python tooling, employs hundreds of people.

Relation to the Web and LAMP Stack

While the classic LAMP stack uses PHP for the application layer, Python has become a major alternative. The LAMP stack — Linux, Apache, MySQL, PHP — represents a traditional open source web deployment model. Python fits into this ecosystem as a PHP replacement, running behind web servers like Apache or Nginx using WSGI (Web Server Gateway Interface). Many modern deployments use Linux, Python, Apache/Nginx, and MySQL/PostgreSQL — functionally equivalent to LAMP.

Community and Governance

Python is governed by the Python Software Foundation (PSF), a non-profit organisation established in 2001. The PSF holds the intellectual property rights to Python, manages the python.org infrastructure,

organises PyCon (the annual Python conference), and awards grants to community projects. Guido van Rossum served as Benevolent Dictator For Life (BDFL) from Python's founding until 2018, when a Steering Council model was adopted. The five-member Steering Council is elected annually by core developers and makes final decisions on language changes through the PEP (Python Enhancement Proposal) process.

Part D — Open Source vs Proprietary Comparison

The closest proprietary alternative to Python as a general-purpose scripting and data language is MATLAB, sold by MathWorks and widely used in engineering and scientific computing. The comparison below evaluates Python against MATLAB across five key dimensions.

Dimension	Python (Open Source)	MATLAB (Proprietary)
Cost	Free — no license required. Unlimited users.	Expensive commercial license. Student editions available but limited.
Security	Source code fully public. Anyone can audit. Patches are transparent.	Source code is closed. Security depends entirely on MathWorks.
Support	Community forums, Stack Overflow, GitHub issues.	Official MathWorks support with SLAs and professional accountability.
Freedom to Modify	Full freedom. Modify interpreter, redistribute, build commercial products.	No freedom. Users cannot modify MATLAB.
Community Control	Governed by elected Steering Council and PSF.	Controlled entirely by MathWorks corporate priorities.
Overall Verdict	Recommended for most use cases. Massive ecosystem, no cost, full freedom.	Suitable only where MathWorks-specific toolboxes are required.

For real-world deployment in most organisations, Python is the clear recommendation. It is free to deploy at any scale, has a massive ecosystem of libraries, is supported by virtually every cloud platform, and is the language of choice for data science, machine learning, and web development. The absence of licensing costs alone makes it attractive — a company deploying Python to a thousand servers pays nothing in language licensing fees.

Contributing to Python is also a meaningful and achievable goal. The CPython repository on GitHub accepts contributions from the public, and the PSF actively encourages new contributors through mentorship programmes like Google Summer of Code. Given everything learned through this audit, contributing to Python represents a participation in the broader project of keeping knowledge open and accessible for everyone.

Part E — Shell Scripts

Five shell scripts were written and tested on Ubuntu 24.04 LTS running on WSL2 (Windows Subsystem for Linux 2) with kernel version 6.6.87.2. All scripts are available in the GitHub repository at https://github.com/kunalcomm/oss-audit-24BAI10443.

Script 1 — System Identity Report

This script displays a formatted welcome screen showing the Linux distribution name, kernel version, current logged-in user, system uptime, current date and time, and the license covering the operating system. It demonstrates the use of variables, command substitution using $(), and the echo command for formatted output.

Shell concepts used: variables (STUDENT_NAME, KERNEL, UPTIME), command substitution ($(uname -r), $(whoami), $(uptime -p), $(lsb_release -d), $(date)), and formatted echo output. All values are gathered dynamically at runtime.

```
admin_kunal@LAPTOP-H4DR3AJS:~/oss-audit$ bash script1.sh
================================
  Open Source Audit – Kunal
================================
Software Chosen : Python
Distro          : Ubuntu 24.04.3 LTS
Kernel          : 6.6.87.2-microsoft-standard-WSL2
User            : admin_kunal
Uptime          : up 1 hour, 22 minutes
Date & Time     : 31 March 2026, 08:18:09
================================
OS License: The Linux Kernel is licensed under GPL v2.
Python is licensed under the PSF License (open source).
================================
admin_kunal@LAPTOP-H4DR3AJS:~/oss-audit$ |
```

Script 2 — FOSS Package Inspector

This script checks whether Python 3 is installed using the dpkg package manager, retrieves version and license information if found, and uses a case statement to print a philosophical note about the package. It demonstrates if-then-else logic, case statements, and piping output through grep and awk.

Shell concepts used: if-then-else (checking dpkg exit code), pipe with grep -E for pattern matching, awk for field extraction, and a case statement mapping package names to philosophy statements.

```
admin_kunal@LAPTOP-H4DR3AJS:~/oss-audit$ bash script2.sh
python3 is installed on this system.

--- Package Details ---
Version : 3.12.3-0ubuntu2
Status  : Installed
License : PSF License (Python Software Foundation)

--- Open Source Philosophy ---
Python: a language built entirely by community volunteers since 1991.
admin_kunal@LAPTOP-H4DR3AJS:~/oss-audit$
```

Script 3 — Disk and Permission Auditor

This script loops through a list of important system directories and reports permissions, owner, group, and disk usage of each. It also specifically checks for Python's installation directory. It demonstrates the for loop, array iteration, and the ls -ld and du commands.

Shell concepts used: bash array (DIRS=(...)), for loop with ${DIRS[@]} expansion, if [-d $DIR] for directory existence check, ls -ld piped to awk for permission extraction, and du -sh for human-readable size output.

```
admin_kunal@LAPTOP-H4DR3AJS:~/oss-audit$ bash script3.sh
================================
  Directory Audit Report
================================
/etc => Permissions: drwxr-xr-x root root | Size: 4.3M
/var/log => Permissions: drwxrwxr-x root syslog | Size: 80M
/home => Permissions: drwxr-xr-x root root | Size: 324K
/usr/bin => Permissions: drwxr-xr-x root root | Size: 94M
/tmp => Permissions: drwxrwxrwt root root | Size: 24K

--- Python Config Directory Check ---
Python dir found: /usr/lib/python3
Permissions: drwxr-xr-x root root
================================
admin_kunal@LAPTOP-H4DR3AJS:~/oss-audit$
```

Script 4 — Log File Analyzer

This script accepts a log file path and optional keyword as command-line arguments, reads the file line by line, counts occurrences of the keyword, and prints a summary with the last five matching lines. Tested with /var/log/dpkg.log — found 1895 occurrences of the keyword 'install'.

Shell concepts used: positional parameters ($1, $2), default value assignment (${2:-error}), if [! -f] for file existence checking, while IFS= read -r for line-by-line reading, arithmetic expansion $((COUNT + 1)) for counting, and grep -i with tail -5 for filtered output.

```
admin_kunal@LAPTOP-H4DR3AJS:~/oss-audit$ bash script4.sh /var/log/dpkg.log install
==================================
  Log File Analyzer
==================================
File    : /var/log/dpkg.log
Keyword : install

Keyword 'install' found 1895 times in /var/log/dpkg.log

--- Last 5 matching lines ---
2025-08-05 16:57:34 status installed software-properties-common:all 0.99.49.2
2025-08-05 16:57:34 status installed ubuntu-wsl:amd64 1.539.2
2025-08-05 16:57:34 status installed libgdk-pixbuf-2.0-0:amd64 2.42.10+dfsg-3ubuntu3.2
2025-08-05 16:57:34 status installed libc-bin:amd64 2.39-0ubuntu8.5
2025-08-05 16:57:34 status installed dbus:amd64 1.14.10-4ubuntu4.1
==================================
admin_kunal@LAPTOP-H4DR3AJS:~/oss-audit$ |
```

Script 5 — Open Source Manifesto Generator

This script interactively asks the user three questions, composes a personalised open source philosophy statement, saves it to a file named after the current user, and displays it on screen. It demonstrates interactive input, string concatenation, file writing, and the date command.

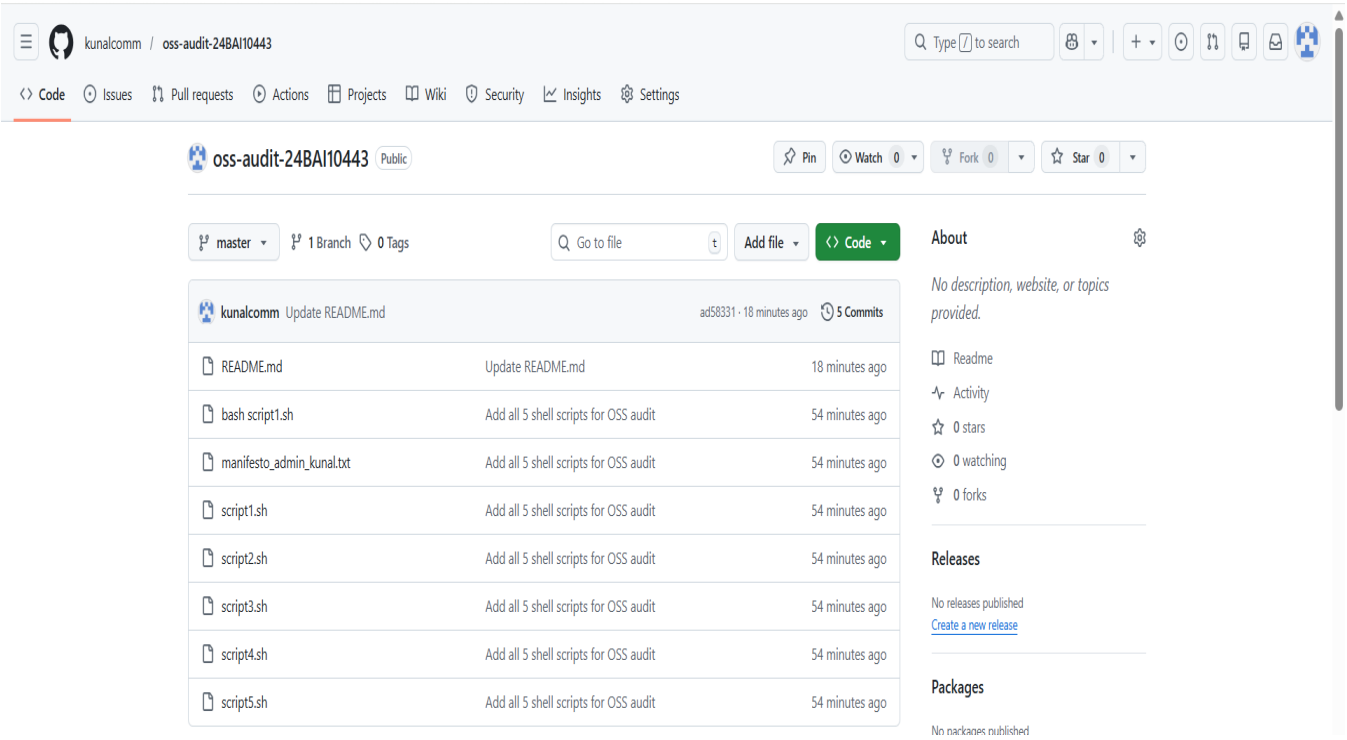
Shell concepts used: read -p for interactive prompting, string interpolation within double-quoted strings, $(date) for current date, $(whoami) for dynamic filename generation, echo with > for file creation, and cat for displaying file contents.

```
admin_kunal@LAPTOP-H4DR3AJS:~/oss-audit$ bash script5.sh
================================
  Open Source Manifesto Generator
================================
Answer three questions to generate your manifesto.

1. Name one open-source tool you use every day: python
2. In one word, what does 'freedom' mean to you? choice
3. Name one thing you would build and share freely: a coding assistant

================================
  Your Manifesto
================================
Open Source Manifesto – admin_kunal – 31 March 2026

I believe in the power of open source. Every day I use python,
a tool built freely by people who chose to share their work with the world.
To me, freedom means choice – and that is exactly what open source represents.
One day I will build a coding assistant and share it freely, because knowledge grows
when it is given away. Standing on the shoulders of giants, I choose
to be a giant for those who come after me.
================================
Manifesto saved to manifesto_admin_kunal.txt
admin_kunal@LAPTOP-H4DR3AJS:~/oss-audit$
```



Conclusion

This audit of Python has been an exercise in understanding software not just as a technical artifact, but as a product of human choices about how knowledge should be shared. Guido van Rossum made a deliberate decision in 1989 to build something useful and give it away freely. That decision, repeated by millions of open source contributors since, has produced the technological infrastructure that the modern world runs on.

Python's PSF License strikes a careful balance between freedom and practicality — permissive enough to encourage commercial adoption, while remaining genuinely open for anyone to use, study, modify, and share. Its governance model, through the PSF and elected Steering Council, demonstrates that software can be maintained sustainably without corporate control. Its ecosystem — from Django to TensorFlow — shows what becomes possible when a good tool is made freely available.

The shell scripts developed alongside this report have demonstrated that open source tools are not just philosophical abstractions. They are practical instruments that live on real systems, in real directories, with real permissions and users. Understanding how Python installs on Linux, how its packages are managed, and how it interacts with the broader system is as important as understanding why it was built.

Open source is not a perfect model. It has tensions — between freedom and sustainability, between community governance and commercial need, between idealism and pragmatism. But as this audit has shown, Python demonstrates that those tensions can be navigated successfully. It is free, powerful, trusted, and improving. That is the open source promise kept.

References

- Van Rossum, G. (1991). Python 0.9.0 Release Notes. python.org
- Python Software Foundation. (2024). PSF License Agreement. python.org/psf/license
- GNU Project. (2024). The Free Software Definition. gnu.org/philosophy/free-sw.html
- Open Source Initiative. (2024). The Open Source Definition. opensource.org/osd
- Raymond, E.S. (1999). The Cathedral and the Bazaar. catb.org/~esr/writings/cathedral-bazaar
- Shotts, W. (2019). The Linux Command Line. linuxcommand.org
- Ubuntu Documentation. (2024). Python in Ubuntu. help.ubuntu.com
- Python Steering Council. (2024). PEP 13 — Python Language Governance. peps.python.org/pep-0013